DEPI Graduation Project

System Analysis & Design

System Under Test: Swag Labs (SauceDemo)

https://www.saucedemo.com

Date: April 23, 2026

System Analysis & Design

4.1 Problem Statement & Objectives

Swag Labs is a deliberately simplified e-commerce demo application used to train and evaluate software testers. While the application is intentionally seeded with known defects (via problem_user, visual_user, etc.), it lacks formal QA documentation — no test plan, no traceability matrix, and no regression suite. This project addresses that gap by delivering a complete test documentation and automation framework.

Project Objectives:

- Produce a fully traceable test suite aligned to the defined functional requirements.
- Expose and document existing defects with reproducible steps, severity, and priority.
- Deliver an automated regression suite using Selenium + Java + TestNG following POM architecture.
- Meet all DEPI graduation project documentation requirements.

4.2 Use Case Diagram Description

Actors

- Guest User – Unauthenticated visitor; limited to the Login page.
- Standard User – Authenticated shopper with full, defect-free access.
- Problem User – Authenticated shopper experiencing seeded UI defects.
- Performance Glitch User – Authenticated shopper with intentional latency.
- System – Responsible for session management, validation, and state transitions.

Primary Use Cases

Use Case ID	Name	Actor	Description
UC-01	Login	All Users	User submits credentials; system validates and grants or denies access.
UC-02	Logout	Authenticated User	User selects Logout from menu; session is terminated.
UC-03	View Product Catalog	Authenticated User	User views all products; applies sorting options.
UC-04	View Product Detail	Authenticated User	User clicks product; system navigates to detail page.
UC-05	Add to Cart	Authenticated User	User adds one or more products; cart badge updates.
UC-06	Remove from Cart	Authenticated User	User removes product from cart or inventory page.
UC-07	Proceed to Checkout	Authenticated User	User initiates checkout from cart page.

Use Case ID	Name	Actor	Description
UC-08	Enter Shipping Info	Authenticated User	User provides first name, last name, postal code.
UC-09	Confirm Order	Authenticated User	User reviews order summary and confirms purchase.
UC-10	Reset App State	Authenticated User	User resets cart and button states via hamburger menu.

4.3 Software Architecture (Observed)

Swag Labs follows a Single-Page Application (SPA) architecture built with React.js. The application state is managed client-side with no visible server-side API calls during standard interactions.

Layer	Technology (Observed)	Notes
Frontend	React.js (SPA)	Component-based UI; dynamic DOM rendering
Routing	Client-side routing	URL changes without full page reload
State Mgmt	localStorage / sessionStorage	Session token stored in browser storage
Hosting	Static web hosting	Served via CDN; no visible backend API

Test Automation Architecture (POM Framework)

- Test Layer: TestNG test classes containing @Test methods and assertions.
- Page Object Layer: LoginPage, InventoryPage, CartPage, CheckoutStepOnePage, CheckoutStepTwoPage, CheckoutCompletePage.
- Base Layer: BaseTest class managing WebDriver lifecycle (@BeforeClass / @AfterClass) and WebDriverManager integration.
- Utilities Layer: ConfigReader (properties file), ScreenshotUtil, WaitHelper (explicit wait wrappers).
- Data Layer: @DataProvider classes and testng.xml for parametric and suite-level execution.

4.4 Database Design & Data Modeling

Note: Swag Labs is a front-end demo with no accessible database. The following represents a logical data model for a real-world equivalent system, included for academic completeness.

Logical Entity Overview

Entity	Key Attributes	Relationships
User	user_id (PK), username, password_hash, account_status, user_type	1 User → M Sessions; 1 User → M Orders
Product	product_id (PK), name, description, price, image_url	M Products ↔ M Orders (via OrderItem)

Entity	Key Attributes	Relationships
Session	session_id (PK), user_id (FK), created_at, expires_at, is_active	M Sessions → 1 User
Cart	cart_id (PK), session_id (FK), created_at	1 Cart → M CartItems
CartItem	cart_item_id (PK), cart_id (FK), product_id (FK), quantity	M CartItems → 1 Cart; M CartItems → 1 Product
Order	order_id (PK), user_id (FK), first_name, last_name, postal_code, total, tax, created_at	1 Order → M OrderItems
OrderItem	order_item_id (PK), order_id (FK), product_id (FK), quantity, price_at_purchase	M OrderItems → 1 Order

4.5 Data Flow & System Behavior

Login Flow

User enters credentials → Frontend validates non-empty inputs → Credentials compared against hardcoded user map → If valid: session token set in localStorage, redirect to /inventory.html → If invalid: error banner displayed on login page.

Add to Cart Flow

User clicks 'Add to cart' → React state updates cart array → Cart badge counter incremented → Button label changes to 'Remove' → Cart data persisted in component state for session duration.

Checkout Flow

User navigates to /cart.html → Clicks 'Checkout' → Routed to /checkout-step-one.html → Submits First Name, Last Name, Postal Code → Validated (non-empty) → Routed to /checkout-step-two.html (order summary with subtotal + 8% tax) → User clicks 'Finish' → Routed to /checkout-complete.html → Cart cleared.

State Diagram: Cart Object

Current State	Trigger	Next State
Empty Cart	User clicks 'Add to cart'	Cart with Items
Cart with Items	User clicks 'Remove' (last item)	Empty Cart
Cart with Items	User clicks 'Checkout'	Checkout In Progress
Checkout In Progress	User confirms order	Order Placed (Cart Cleared)
Any State	User clicks 'Reset App State'	Empty Cart
Any State	Session expires / Logout	Session Terminated

4.6 UI/UX Design – Screen Inventory

Screen	URL	Key UI Elements
Login Page	/	Username field, Password field, Login button, Error banner
Inventory Page	/inventory.html	Product grid (6 items), Sort dropdown, Cart icon, Hamburger menu
Product Detail Page	/inventory-item.html	Product image, Name, Description, Price, Add to Cart / Back button
Cart Page	/cart.html	Cart item list, Quantity, Remove button, Continue Shopping, Checkout
Checkout Step 1	/checkout-step-one.html	First Name, Last Name, Postal Code fields, Cancel, Continue
Checkout Step 2	/checkout-step-two.html	Item list, Price breakdown (subtotal + tax + total), Finish button
Order Confirmation	/checkout-complete.html	Success message, Pony Express image, Back Home button

UI/UX Notes & Known Defects

- Color scheme: Primary #E2231A (red), Background #FFFFFF, Navigation #132322 (dark green).
- Typography: Sans-serif throughout; consistent sizing for headings vs body text.
- problem_user defects: Product images render incorrectly; Last Name field cannot be populated on checkout; Add to Cart adds incorrect products.
- visual_user defects: Visual layout inconsistencies suitable for visual regression testing.

4.7 System Deployment & Integration

Component	Detail
Frontend Framework	React.js (Single Page Application)
Hosting	Static CDN — Sauce Labs managed (saucedemo.com)
Browser Compatibility	Chrome, Firefox, Edge (latest versions)
Automation Stack	Java 11+, Selenium WebDriver 4.x, TestNG 7.x, Maven 3.x, WebDriverManager
Version Control	GitHub (public repository)
Build Tool	Maven — mvn clean test for full suite execution
Reporting	TestNG HTML Reports / Extent Reports (optional)

4.8 Testing & Validation Plan

Test Level	Type	Tool / Approach
System Testing	Functional test cases (FR-01 to FR-17)	Manual (Excel-based test cases)

Test Level	Type	Tool / Approach
Regression	Automated regression suite	Selenium + TestNG POM framework
UAT	Checklist-based walkthrough	Manual — standard_user persona
Performance	Page load time observation	Browser DevTools / Network tab
Security	Session handling, direct URL access	Manual — NFR-06, NFR-07
Compatibility	Cross-browser rendering	Chrome, Firefox, Edge
Exploratory	Unscripted investigation	Manual — all user types